Tribhuvan University
Institute of Engineering
Pulchowk Campus
Department of Electronics and Computer Engineering

Deadlock

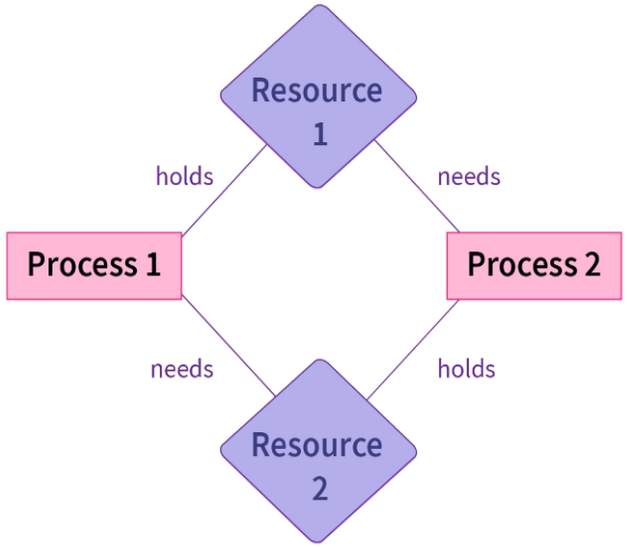
Santosh Giri
Asst. Professor, IOE, Pulchowk Campus.

Deadlock: Basics

- A process in operating systems uses different resources and uses resources in the following way:
  1) Requests a resource
  2) Use the resource
  3) Releases the resource
- Deadlock is a common problem in multi-processing, where several processes share a specific type of mutually exclusive resource known as a soft lock or software.
- When two or more processes hold some resources and wait for resources held by other(s), then deadlock is occurred.

Deadlock: Basics

Example



- Here, Process 1 is holding Resource 1 and waiting for Resource 2 which is acquired by Process 2. And Process 2 is waiting for Resource 1 and holding Resource 2.
- Because both processes are waiting, none of them will ever cause any of the events that could wake up any of the other members in the set, and both processes continue to wait forever.
- i.e. none of the processes can: run, release resources and be awakened

Deadlock: Basics

Resources:

- Resources can be I/O devices, memory spaces, processors, disks, etc.
- A process may need multiple resources to accomplish the task.

Types:

1. **Pre-emptive resources**
   - These resources can be taken away from the processes owning it with no ill effects
   - E.g.: Memory
2. **Non Pre-emptive Resources**
   - Resources that cannot be taken away from their current owner without causing the computation to fail.
   - E.g.: Scanner, printer, etc.

*In general, deadlocks involve **Non** **Pre-emptive** resources*

Principle of Deadlock

Deadlock can arise if the following four conditions hold simultaneously (*Necessary conditions of deadlock*)

- **Mutual Exclusion**
  - ✓ One or more than one resource are non-shareable (Only one process can use at a time)
- **Hold and Wait**
  - ✓ A process is holding at least one resource and waiting for resources.
- **No Preemption**
  - ✓ A resource cannot be taken from a process unless the process releases the resource.
- **Circular Wait**
  - ✓ A set of processes are waiting for each other in a circular form.

- All four of these conditions must be present for a deadlock to occur. If one of them is absent, no deadlock is possible.

Methods of Deadlock Handling

- **Deadlock Prevention**
  - ✓ by structurally negating one of the four necessary conditions.
- **Deadlock Ignorance(Ostrich's Algorithm)**
  - ✓ Maybe if you ignore it, it will ignore you.
- **Deadlock Avoidance (Banker's Algorithm)**
  - ✓ Dynamic avoidance by careful resource allocation
- **Deadlock Detection and Recovery**
  - ✓ Let them occur, detect them, and take action.

Deadlock Prevention

- It involves techniques to attack the four conditions that may cause deadlock (Coffmann Conditions).

1. **Prevention from Mutual Exclusion:**
   - The resources should not be assigned to a process until absolutely necessary.

2. **Prevention Wait and Hold State:**
   - The process should request its required resources at the beginning.
   - If every resource is available, the process runs or else it waits but does not hold any resources.
   - This way hold and wait condition is eliminated but it will lead to low device utilization.

   For example

   - if a process requires a printer at a later time and we have allocated a printer before the start of its execution printer will remained blocked till it has completed its execution.

Deadlock Prevention

3. Preventing the non-preemption condition:

- Preempt resources from a process when resources are required by another high-priority process.

4. Preventing Circular Wait:

- Each resource will be assigned a numerical number. And a process can request the resources only in increasing order of numbering.

For example:

- If the P1 process is allocated R5 resources, now next time if P1 asks for R4, R3 lesser than R5 such a request will not be granted, only requests for resources more than R5 will be granted.

Deadlock Ignorance(Ostrich's Algorithm)

- The simplest approach to ignore the deadlock is the ostrich algorithm: stick your head in the sand and pretend there is no problem.
- People react to this strategy in different ways. Mathematicians find it unacceptable and say that deadlocks must be prevented at all costs.
- Engineers ask how often the problem is expected, how often the system crashes for other reasons, and how serious a deadlock is.
- If deadlocks occur on average once every five years, but the system crashes due to hardware failures and operating system bugs occur once a week, most engineers would not be willing to pay a large penalty in performance or convenience to eliminate deadlocks.

Deadlock Avoidance :The Banker's Algorithm

- We cannot always assume that a process will ask for all its required resources at once.
- The system must decide whether granting a resource is safe or not and make the allocation if it is safe.
- So, we need an algorithm to avoid deadlock by making the right choices all the time given that some information is available in advance.
- When working with a banker's algorithm, we must know about the following things:
  - ✓ How much each process can request for each resource in the system? It is denoted by the [MAX] request.
  - ✓ How much each process is currently holding each resource in a system? It is denoted by the [ALLOCATED] resource.
  - ✓ How many numbers of each resource are currently available in the system? It is denoted by the [AVAILABLE] resource.
  - ✓ The remaining need of the system. It is denoted by the [Remaining Need].
  - ✓ Remaining NEED =[MAX] – [ALLOCATED]
  - ✓ FREE = [TOTAL RESOURCE] – [TOTAL ALLOCATED]

The Banker's Algorithm:

for multiple instances of multiple resources (Example 1)

Total resources
A=10, B=5 , C=7
Remaining need= Maximum need – Allocation
Available = Total available –Total Allocation

Process	Allocation			Maximum Need		
	A	**B**	**C**	**A**	**B**	**C**
P1	0	1	0	7	5	3
P2	2	0	0	3	2	2
P3	3	0	2	9	0	2
P4	2	1	1	4	2	2
P5	0	0	2	5	3	3

The Banker's Algorithm: Example 1

Calculate Remaining need= Maximum need - Allocation

Process	Allocation			Maximum Need			Remaining need		
	A	**B**	**C**	**A**	**B**	**C**	**A**	**B**	**C**
P1	0	1	0	7	5	3	7	4	3
P2	2	0	0	3	2	2	1	2	2
P3	3	0	2	9	0	2	6	0	0
P4	2	1	1	4	2	2	2	1	1
P5	0	0	2	5	3	3	5	3	1
Total	7	2	5						

The Banker's Algorithm: Example 1

Calculate Available = Total resources- total Allocation i.e.

Total resources: A=10, B=5 , C=7

Total allocation: A=**7**, B=**2** , C=**5**

So Available : for A=10-7=**3**, for B=5-2=**3** and for C=7-5=**2**

Process	Allocation			Maximum Need			Remaining need		
	A	B	C	A	B	C	A	B	C
P1 (4)	0	1	0	7	5	3	7	4	3
P2(1)	2	0	0	3	2	2	1	2	2
P3 (5)	3	0	2	9	0	2	6	0	0
P4 (2)	2	1	1	4	2	2	2	1	1
P5 (3)	0	0	2	5	3	3	5	3	1
Total	**7**	**2**	**5**						

No Deadlock so the Safe Sequence is: P2 → P4 → P5 → P1 → P3

Available		
A	B	C
3	3	2
2	0	0
5	3	2
2	1	1
7	4	3
0	0	2
7	4	5
0	1	0
7	5	5
3	0	2
10	5	7

The Banker's Algorithm: Example2

Total resources

R1=15, R2=9 , R3=5

Process	**Allocation**			**Maximum**		
	R1	R2	R3	R1	R2	R3
P1	3	0	1	3	2	2
P2	5	4	1	6	8	2
P3	2	2	0	3	2	4
P4	2	1	3	4	2	3

The Banker's Algorithm: Example3 (2080 R)

Total resources: R0 = 6, R1 = 4, R2 = 4, R3 = 2

	R0	R1	R2	R3			R0	R1	R2	R3
P0	2	0	1	1		P0	3	2	1	1
P1	1	1	0	0		P1	1	2	0	2
P2	1	1	0	0		P2	1	1	2	0
P3	1	0	1	0		P3	3	2	1	0
P4	0	1	0	1		P4	2	1	0	1

A safe sequence of execution for this system is: **P2 → P3 → P4 → P0 → P1**

Deadlock Detection & Recovery

Deadlock Detection

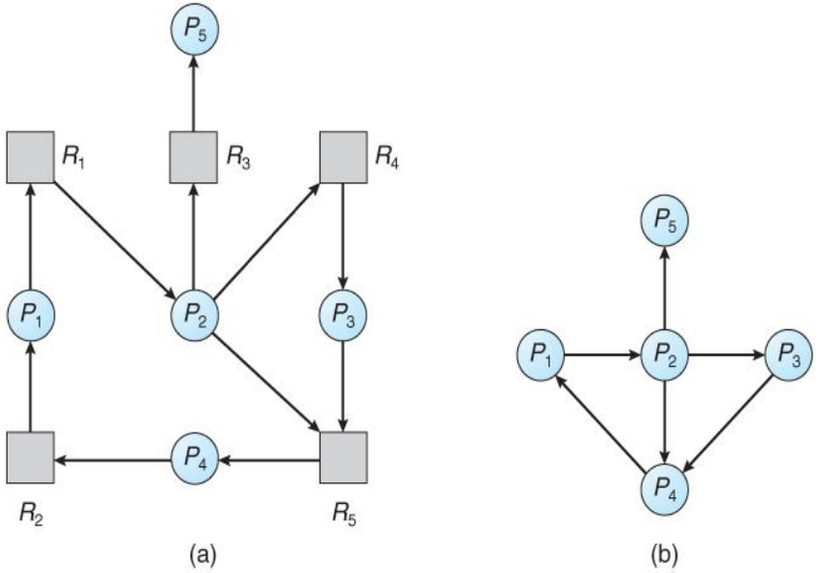
- ✓ Deadlock Detection for Single Instance of Each Resource Type
- ✓ Deadlock Detection for Several Instances of a Resource Type

Recovery

- ✓ Take manual intervention.
- ✓ Terminate one or more processes involved in the deadlock
- ✓ Preempt resources.

Deadlock Detection: Single Instance of Each Resource Type

- If each resource category has a single instance, then we can use a variation of the resource-allocation graph known as a **wait-for graph**.
- A **wait-for graph** can be constructed from a resource-allocation graph by eliminating the resources and collapsing the associated edges, as shown in the figure below.
- An arc from process Pi to Pj in a wait-for graph indicates that process Pi is waiting for a resource that process Pj is currently holding.

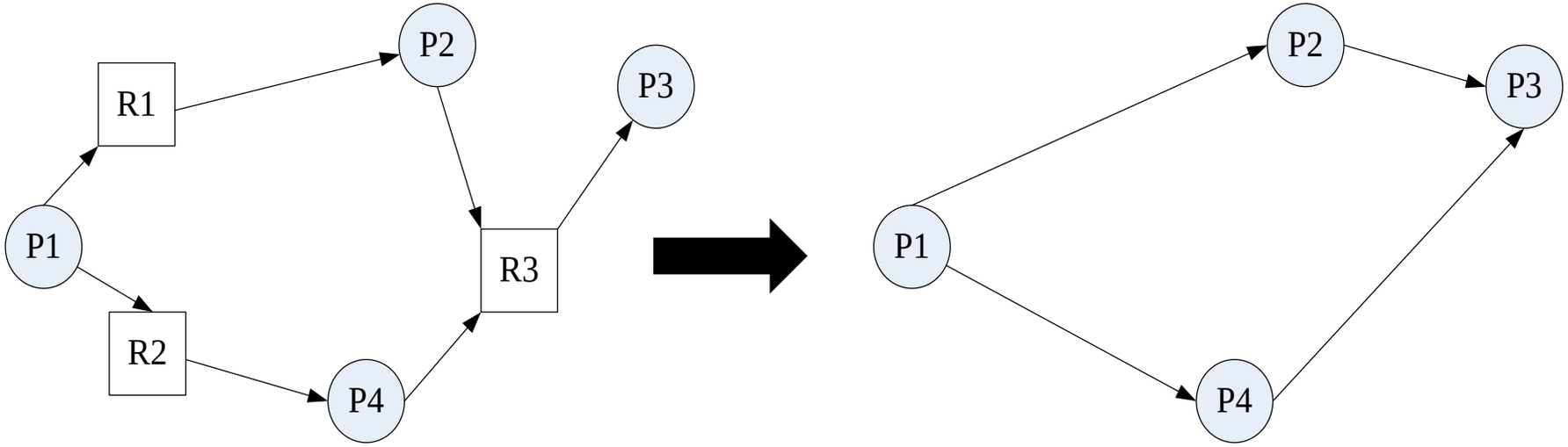


(a) Resource allocation graph. (b) Corresponding wait-for graph

- This algorithm must maintain the wait-for graph and periodically search it for cycles and cycles in the wait-for graph indicate deadlocks.

Deadlock Detection: Single Instance of Each Resource Type

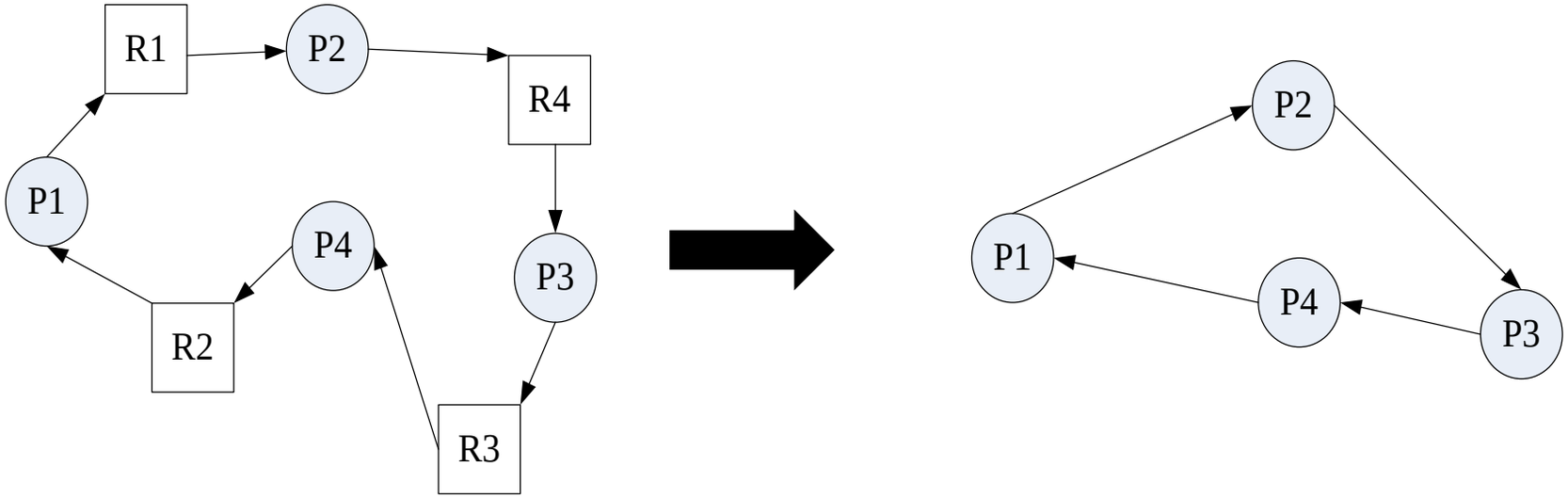
Class Work: Draw the wait-for graph



Non-cyclic graph, so no deadlock

Deadlock Detection: Single Instance of Each Resource Type

Class Work: Draw the wait-for graph



cyclic graph, so deadlock is present

Deadlock Detection: Several Instances of a Resource Type

The detection algorithm outlined here is essentially the same as the Banker's algorithm, with two subtle differences:

- In the beginning, the Banker's Algorithm sets Finish[i] to false for all i. The algorithm presented here sets Finish[i] to false only if Allocation[i] is not zero i.e. If Allocation [i] is zero, the algorithm sets Finish[i] to true.
  - ✓ This is essentially assuming that IF all of the other processes can finish, then this process can finish also.
  - ✓ Furthermore, this algorithm is specifically looking for which processes are involved in a deadlock situation, and a process that does not have any resources allocated cannot be involved in a deadlock, and so can be removed from any further consideration.
- The Banker's Algorithm says that if Finish[i] == true for all i, there is no deadlock whereas, this algorithm is more specific, by stating that if Finish[i] == false for any process Pi, then that process is specifically involved in the deadlock which has been detected.

Deadlock Recovery

There are three basic approaches to recovery from deadlock:

- Take manual intervention
  - ✓ Inform the system operator, and allow him/her to take manual intervention.
- Terminate processes
  - ✓ Terminate a single process
  - ✓ Terminate all process
- Resource Preemption
  - ✓ Preemption
  - ✓ Rollback

Deadlock Recovery

Terminate process

Two basic approaches, both of which recover resources allocated to terminated processes:

- ✓ **Terminate all processes** involved in the deadlock. This definitely solves the deadlock but at the expense of terminating more processes than would be absolutely necessary.
- ✓ **Terminate processes one by one** until the deadlock is broken. This is more conservative but requires doing deadlock detection after each step.

In the latter case, there are many factors that can go into deciding which processes to terminate next:

- ✓ Process priorities.
- ✓ How long the process has been running, and how close it is to finishing.
- ✓ How many and what type of resources is the process holding? (Are they easy to preempt and restore?)
- ✓ How many more resources does the process need to complete?
- ✓ How many processes will need to be terminated?
- ✓ Whether or not the process has made non-restorable changes to any resource.

Deadlock Recovery

Resource Preemption

- Preemption
  - ✓ Here, we transfer resources from one process to the process that needs them to finish its execution, and once that process is done, the resource is released.
- RollBack
  - ✓ When we identify Deadlock, we must roll back all allocations and return to the previous safe state

When preempting resources to relieve deadlock, there are three important issues to be addressed:

- ✓ **Selecting a victim**: Deciding which resources to preempt from which processes involve many of the same decision criteria outlined in terminating a process.
- ✓ **Rollback**: Ideally one would like to roll back a preempted process to a safe state prior to the point at which that resource was originally allocated to the process. Unfortunately, it can be difficult or impossible to determine what such a safe state is, and so the only safe rollback is to roll back all the way back to the beginning. (I.e. abort the process and make it start over.)
- ✓ **Starvation**: How do you guarantee that a process won't starve because its resources are constantly being preempted? One option would be to use a priority system and increase the priority of a process every time its resources get preempted. Eventually, it should get a high enough priority that it won't get preempted anymore.

Assignment 4

Q. The Banker's Algorithm: for multiple instances "**Example 1"** and **"Example 2"**
Q. Write short notes on:

1. Two Phase Locking
2. Livestock
3. Communication Deadlock

Q. Differentiate Between

- Starvation & Deadlock
- Deadlock & Indefinite Postponement